



**INSTITUTE FOR ADVANCED COMPUTING AND
SOFTWARE DEVELOPMENT, AKURDI, PUNE**

“Organic Bazar”

PG-DAC March 2025

Submitted By:

Group No: 22

Roll No.	Name of Student
252138	Bhushan Sunil Sayankar
252169	Mayank Panbude

Mrs. Monika Sindhikar
Project Guide

Mr. Prashant Deshpande
Centre Coordinator

ABSTRACT

Organic Bazar is a microservices-based e-commerce platform that bridges the gap between local organic farmers and health-conscious customers. Built using Spring Boot Microservices for the backend and React JS for the frontend, the application allows for modular, scalable development with independent services for authentication, user management, products, orders, and payments.

Each microservice handles a specific responsibility and communicates via REST APIs, registered and discovered through Eureka. A central API Gateway routes requests, manages authentication with JWT, and ensures secure access for three main roles: Admin, Farmer, and Customer.

Admins manage users and categories; Farmers upload and manage products; Customers browse, order, and pay securely. The frontend, built in React, provides a responsive and role-based interface. Backend services use Spring Data JPA and MySQL for persistence.

The project promotes healthy living and supports sustainable farming. By separating services and enabling independent scaling and deployment, Organic Bazar provides high performance, fault tolerance, and ease of maintenance, making it a robust real-world application for organic commerce.

ACKNOWLEDGEMENT

I take this occasion to thank God, almighty for blessing us with his grace and taking our endeavor to a successful culmination. I extend my heartfelt thanks to our esteemed guide, **Mrs. Monika Sindhikar** for providing me with the right guidance and advice at the crucial juncture and showing me the right way. I sincerely thank our respected Centre Co- Ordinator, Mr. Prashant Deshpande, for allowing us to use the available facilities. I would also like to thank the other faculty members at this occasion. Last but not least, I would like to thank my friends and family for the support and encouragement they have given me during our work.

Bhushan Sunil Sayankar (250241220048)

Mayank Panbude (250241220111)

TABLE OF CONTENTS

Sr.No	Description	Page No.
1	Introduction	5
3	SRS	11
4	Diagrams	15
4.1	ER Diagram	15
4.2	Use Case Diagram	16
4.3	Data Flow Diagram	17
4.4	Activity Diagram	19
4.5	Class Diagram	20
4.6	Sequence Diagram	21
5	Database Design	22
6	Snapshots	25
7	Conclusion	33
8	Future Scope	34
9	References	35

1. INTRODUCTION

Organic Bazar: In today's digital era, the demand for organic and chemical-free food products is rapidly increasing as people grow more health-conscious and environmentally aware. However, local organic farmers often struggle to reach consumers directly due to lack of access to digital platforms. To address this challenge, Organic Bazar has been developed as a full-stack e-commerce application that connects organic farmers directly with customers.

The application is built using a microservices architecture with Spring Boot on the backend and React JS on the frontend. It provides a structured and scalable solution by dividing the system into independent, modular services such as Authentication, User Management, Product Catalog, Orders, and Payments. Each service is loosely coupled, enabling independent deployment, scalability, and fault isolation.

The platform supports multiple user roles: Admin, Farmer, and Customer. Admins manage users, categories, and oversee the system. Farmers can register, list their products, and manage stock. Customers can browse categories, add products to cart, place orders, and make secure payments. Authentication is handled using JWT tokens, and services communicate through REST APIs, managed centrally by an API Gateway and Eureka Service Discovery.

Organic Bazar is not just a software solution—it is a step toward empowering local farmers, encouraging organic consumption, and building a sustainable digital marketplace. With its clean architecture and real-world usability, this project reflects modern software design practices and social impact through technology.

1.1 Purpose

The Primary purpose of the Organic Bazar project is to digitally empower local organic farmers by providing them with a platform to directly sell their produce to customers without the interference of middlemen. It aims to create a transparent, efficient, and scalable online marketplace for organic products where:

- Farmers can showcase and manage their products independently.
- Customers can access fresh, chemical-free food directly from verified sources.
- Admins can regulate platform activities, ensuring fairness and quality.

From a technical standpoint, the project also serves as a practical implementation of Microservices Architecture, demonstrating how complex applications can be modularized into small, manageable, and independently deployable services using Spring Boot. It highlights how React JS can be used to develop a modern, responsive frontend that interacts with multiple backend services seamlessly.

Ultimately, the project promotes sustainability, fair trade, and technology-driven solutions to real-world problems in the agriculture and food industry.

1.2 Project Background

The global shift toward healthier lifestyles has increased the demand for organic products. However, in many parts of the world—especially rural regions—organic farmers face difficulties in reaching a broader customer base due to a lack of digital access, proper platforms, and technical support. At the same time, urban customers struggle to find reliable sources of organic products at fair prices.

To bridge this gap, the **Organic Bazar** project was conceptualized as a digital solution that connects organic farmers directly to end consumers through a secure and scalable e-commerce platform. The idea is to cut out intermediaries, ensure better profit margins for farmers, and provide customers with fresh, chemical-free products from trusted sources.

From a development standpoint, the project leverages **modern software architecture practices**, specifically **microservices**, to address scalability, maintainability, and distributed system challenges. Technologies like **Spring Boot**, **Spring Cloud**, **Eureka**, **Spring Security**, and **React JS** are used to create a modular and robust platform that supports multiple user roles and complex workflows, such as product management, order processing, and secure payments.

The project was developed with real-world usability in mind, ensuring that it can be extended or deployed in live environments to support local farmer markets, agricultural co-operatives, and organic startups.

1.3 Goal of the Project

The primary goal of the **Organic Bazar** project is to build a secure, scalable, and user-friendly e-commerce platform that bridges the gap between organic farmers and consumers. The project aims to digitally empower local farmers by providing them with direct access to a wider customer base without the involvement of intermediaries, thereby increasing their profitability and visibility. Another major goal is to promote healthy and sustainable living by enabling consumers to purchase fresh, organic products from reliable sources.

From a technical standpoint, the project focuses on implementing a modular **microservices architecture** using Spring Boot to achieve independent development, deployment, and scaling of backend services. It also aims to ensure smooth and secure user interactions through **JWT-based authentication**, role-based access control, and a centralized API Gateway. The use of **React JS** for the frontend serves the goal of providing a responsive, intuitive, and role-specific interface for Admins, Farmers, and Customers. Additional goals include enabling streamlined order placement, secure payment processing, and admin-level controls to manage users, products, and categories effectively. Ultimately, the project is designed with real-world scalability and deployment readiness in mind.

2. OVERALL DESCRIPTION

Proposed Methodology:

The **Organic Bazar** project follows a modular and scalable architecture based on microservices, utilizing **Spring Boot** for backend services and **React JS** for the frontend interface. The system is designed to separate concerns, enhance maintainability, and support independent deployment of services.

1. Microservices Architecture

Each core module of the application (such as user management, product management, order handling, and payment processing) is implemented as an independent microservice. This architecture ensures that services can be developed, tested, and deployed separately, leading to higher scalability and fault isolation.

2. Backend – Spring Boot Microservices

- **Spring Boot** is used to develop RESTful APIs for each microservice.
- Services interact with a central **MySQL** database (or distributed databases if needed).
- Key microservices include:
 - **Authentication Service** (JWT-based login, role-based access)
 - **User Service** (Admin, Farmer, Customer roles)
 - **Product Service** (Categories, Organic Products)
 - **Cart and Order Service**
 - **Payment Service**

3. Frontend – React JS

- **React JS** is used to create a dynamic and responsive user interface.
- Role-based views are implemented:
 - **Admin Panel** – manage users, categories, products, and orders.
 - **Farmer Dashboard** – upload products, manage listings.
 - **Customer Portal** – browse products, add to cart, place orders.

4. API Gateway and Service Communication

- An **API Gateway** acts as a single entry point for all client requests.
- Internal communication between microservices is handled using **REST API calls** or **Feign Clients**.
- **Eureka Server** can be used for service discovery.

5. Security and Authentication

- **JWT (JSON Web Tokens)** are used for secure, stateless authentication.
- Role-based access control ensures that each user has appropriate access privileges.

6. Database Management

- The system uses **MySQL** for structured data storage.
- Proper entity relationships are maintained across tables for users, products, orders, and payments.

7. Development Tools

- **Spring Boot, Spring Cloud, React JS, Node.js** (for build tools), **MySQL, Postman**, and **GitHub** are used for the development and version control of the project.

8. Testing and Validations

- Each module is tested independently using **unit tests** and **Postman** for API validation.
- Frontend components are tested using browser dev tools and console debugging.

3. SOFTWARE REQUIREMENT SPECIFICATION

The functional requirements for the **Organic Bazar** platform define the core features and capabilities necessary to meet the needs of customers, farmers, and administrators. These requirements form the foundation of the development process, ensuring that the final system aligns with business objectives while providing a seamless and efficient online marketplace for organic products. By clearly outlining these requirements, the project ensures that all stakeholders—ranging from end-users seeking fresh produce to administrators managing inventory and transactions—experience an intuitive, reliable, and value-driven platform. This framework not only supports day-to-day operations but also fosters transparency, trust, and scalability in the organic e-commerce ecosystem.

3.1 Functional Requirements

1. User Management

- User registration and login with role-based access (Admin, Farmer, Customer).
- Profile management (update personal details, change password).

2. Product Management

- Farmers can add, edit, and delete their products.
- Admin can manage all products and product categories.
- Customers can view and search products by category or name.

3. Cart and Order Management

- Customers can add/remove products from the cart.
- Place orders with order summary and details.
- Track order status (Pending, Confirmed, Shipped, Delivered).

4. Payment Processing

- Secure payment gateway integration for online payments.
- Order confirmation upon successful payment.

5. Admin Functionalities

- Manage users (approve/reject farmers).
- View sales reports and transactions.
- Manage product categories and site content.

3.2 Non - Functional Requirements

1. Performance Requirements

- The system should handle at least 500 concurrent users.
- Response time should be under 3 seconds for any operation.

2. Security Requirements

- Passwords stored in encrypted format.
- HTTPS protocol for secure data transmission.
- Role-based authorization to restrict access to specific functionalities.

3. Reliability and Availability

- System uptime of 99.5%.
- Data backup performed daily.

4. Scalability

- Microservices architecture ensures horizontal scaling for high traffic.

3.3 Software Requirements

1. Frontend

- Technology: React JS
- Libraries: Axios, Bootstrap/Tailwind CSS

2. Backend

- Technology: Spring Boot (Java)
- Architecture: Microservices with REST APIs
- Frameworks & Tools: Spring Data JPA, Spring Security, Spring Cloud

3. Database

- Type: MySQL
- ORM: Hibernate/JPA

4. Development Tools

- IDE: IntelliJ IDEA / Eclipse (Backend), VS Code (Frontend)
- Version Control: Git & GitHub
- Build Tools: Maven

5. Deployment Environment

- Server: Apache Tomcat

3.4 Hardware Requirements

1. Development Machine

- Processor: Intel i5 or above
- RAM: 8GB or more
- Storage: 500GB HDD / SSD

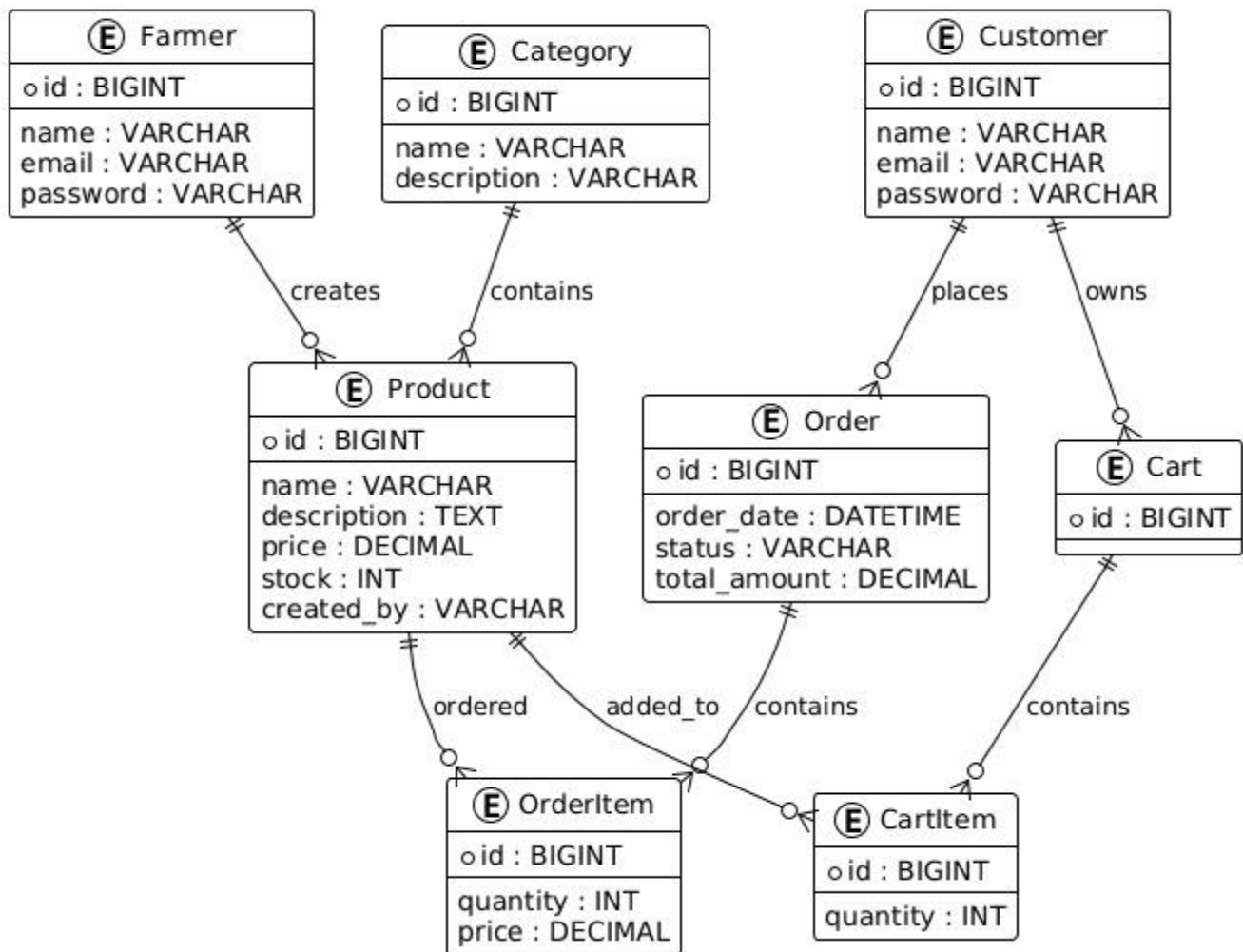
2. Server

- Processor: Quad-Core CPU
- RAM: 16GB or above
- Storage: 1TB SSD
- Internet: High-speed broadband

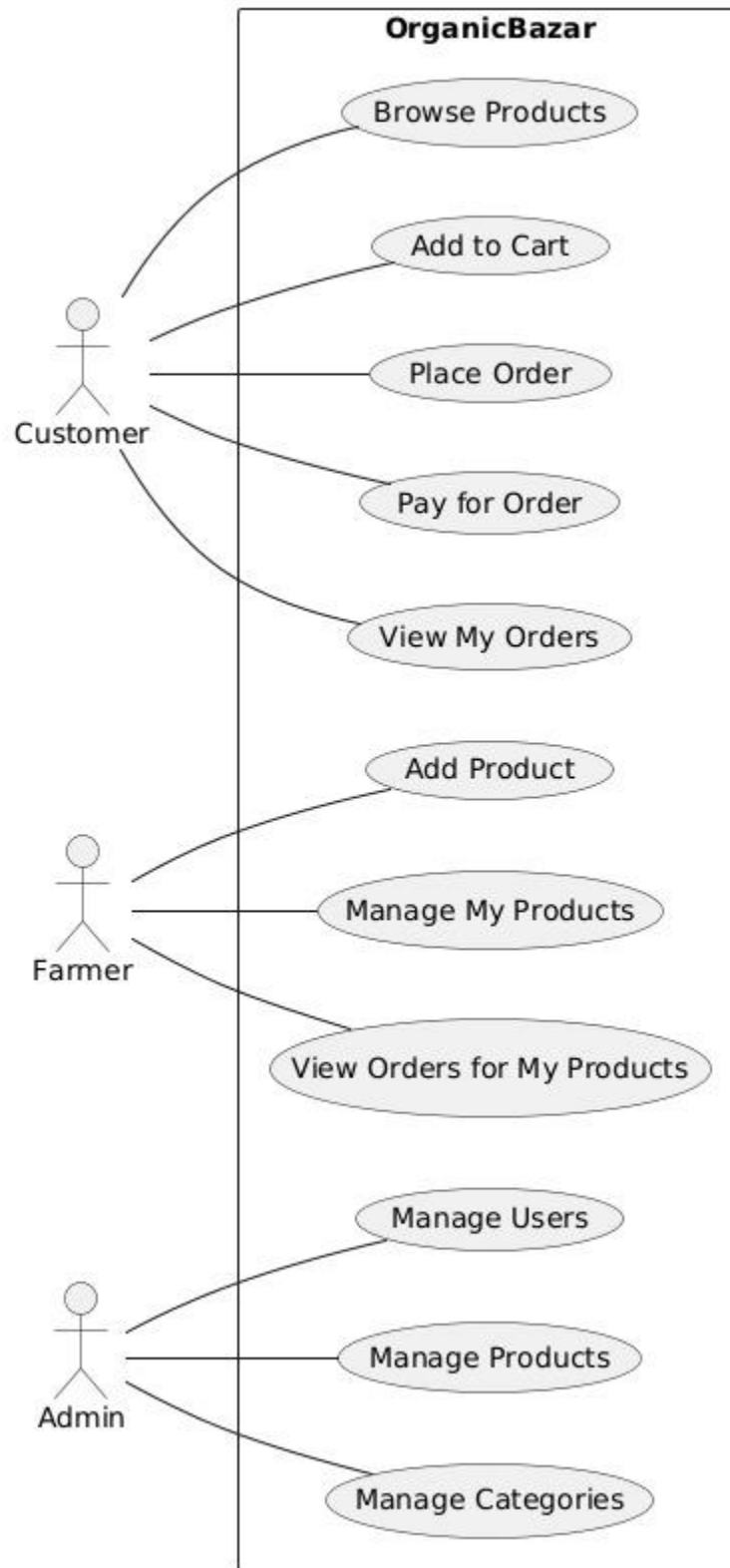
4. DIAGRAMS

4.1 Entity Relationship Diagram :

OrganicBazar ER Diagram



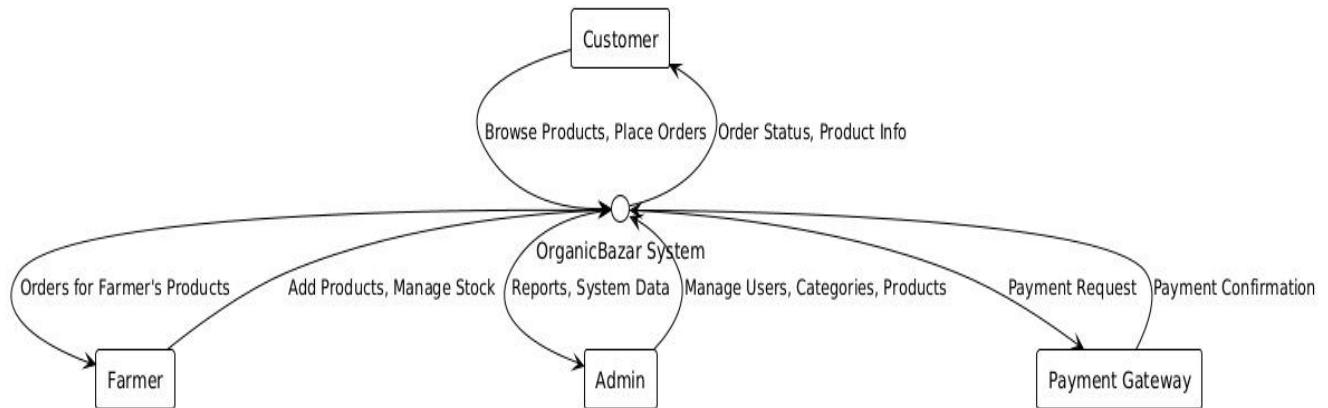
4.2 Use Case Diagram :



4.3 Data Flow Diagram :

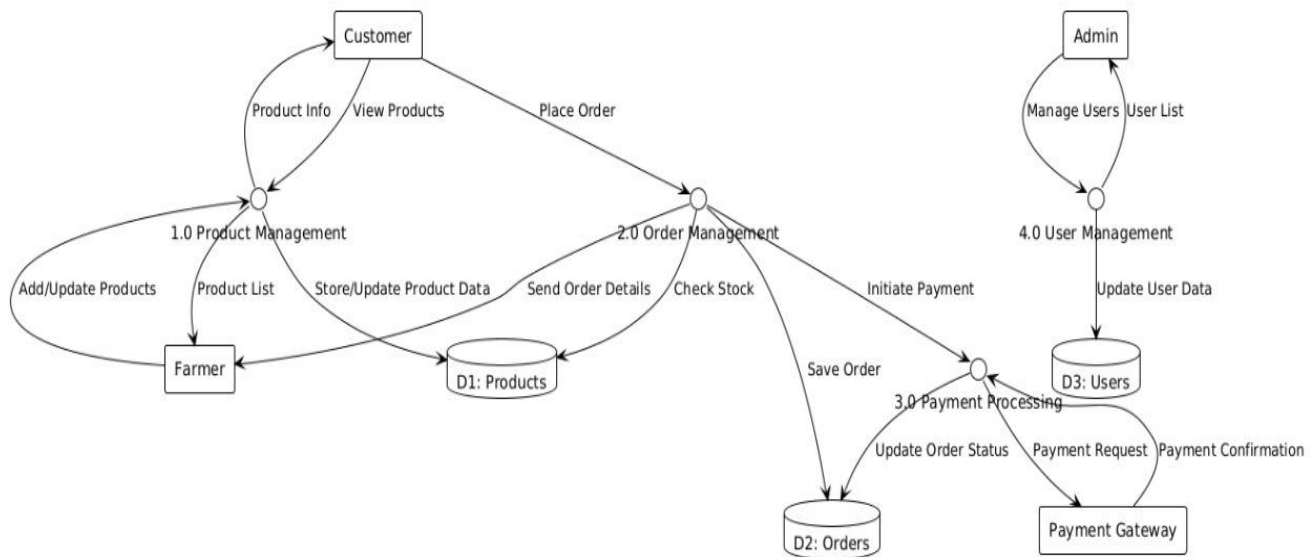
Level 0 :

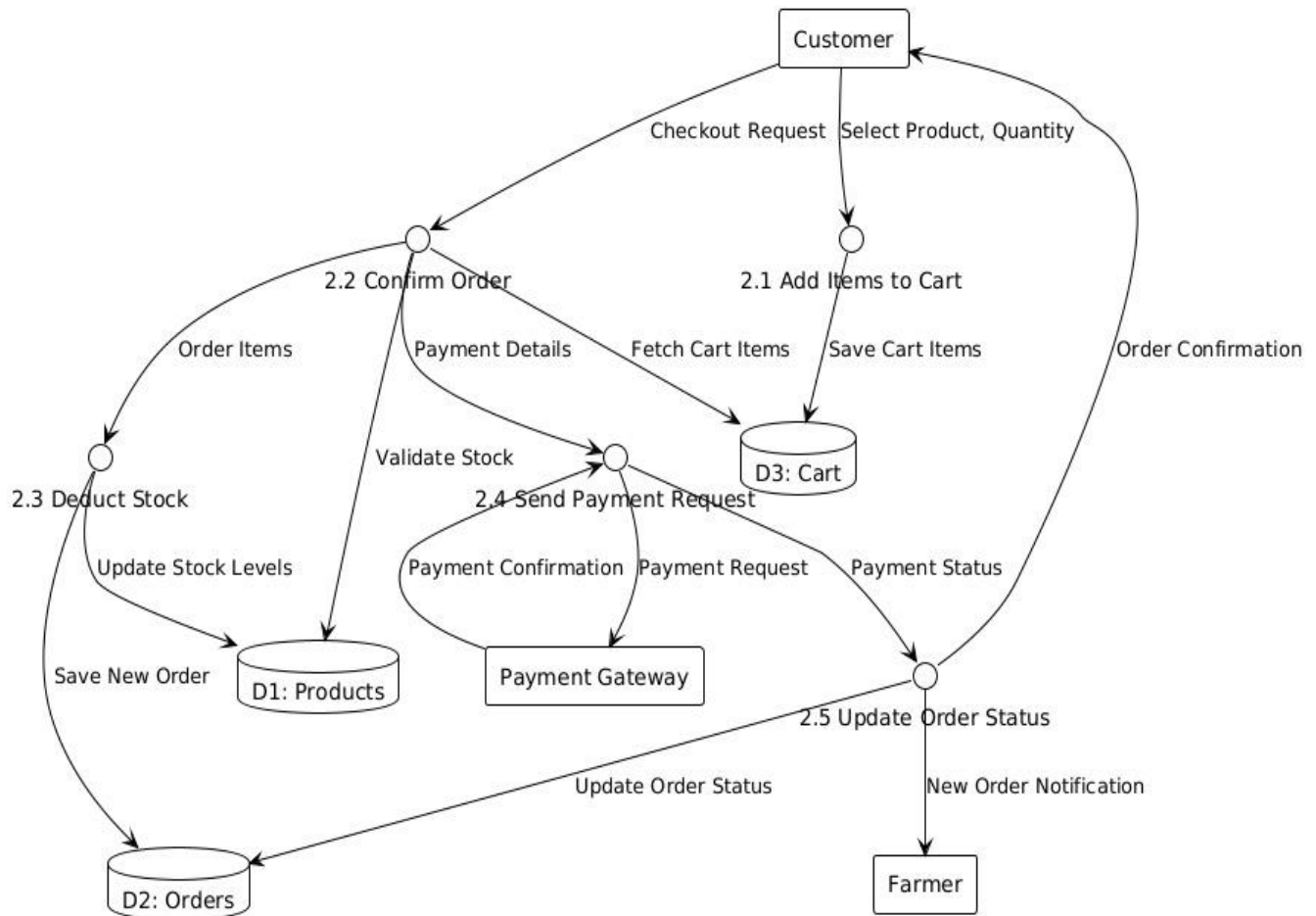
OrganicBazar DFD - Level 0



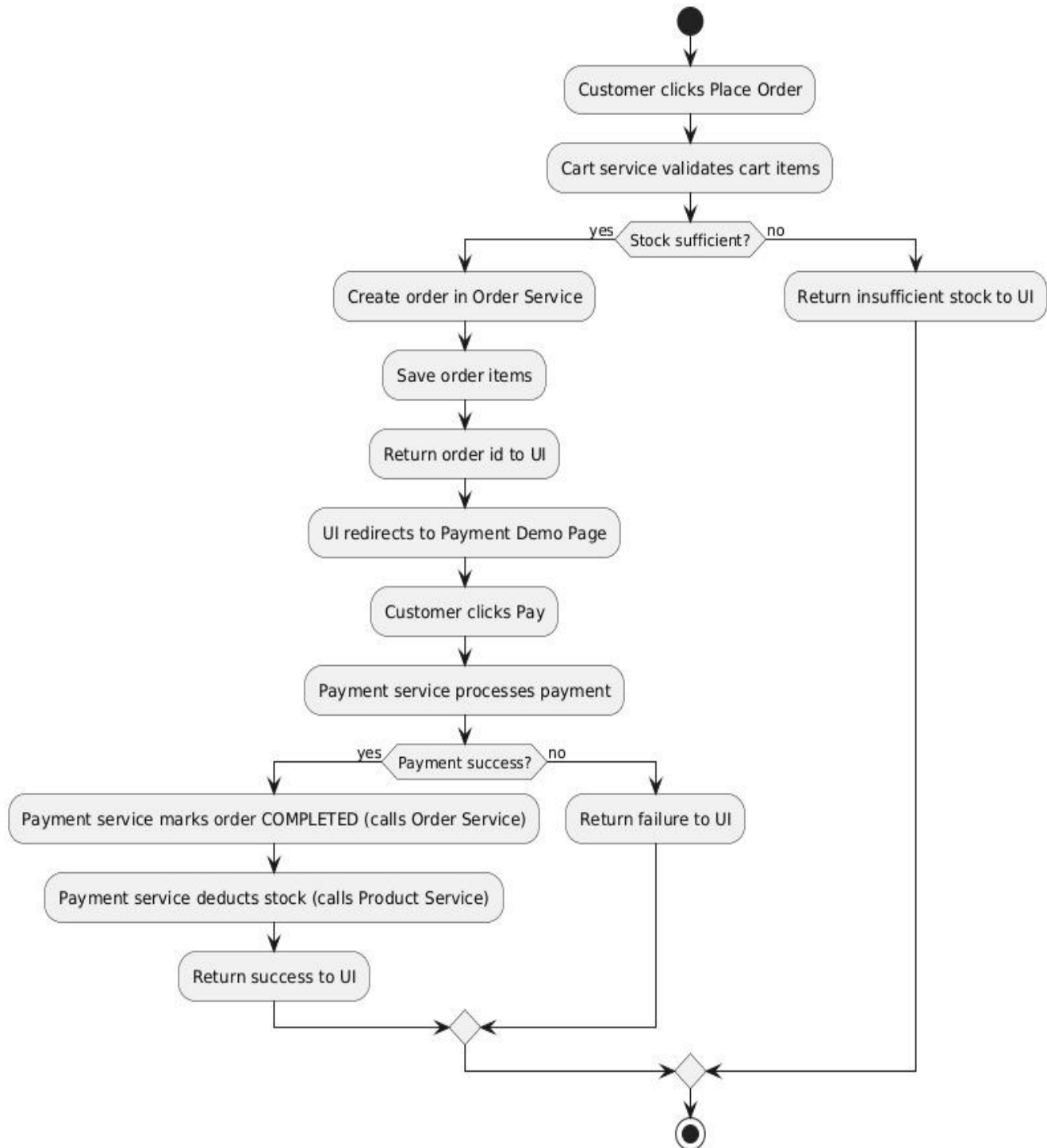
Level 1 :

OrganicBazar DFD - Level 1



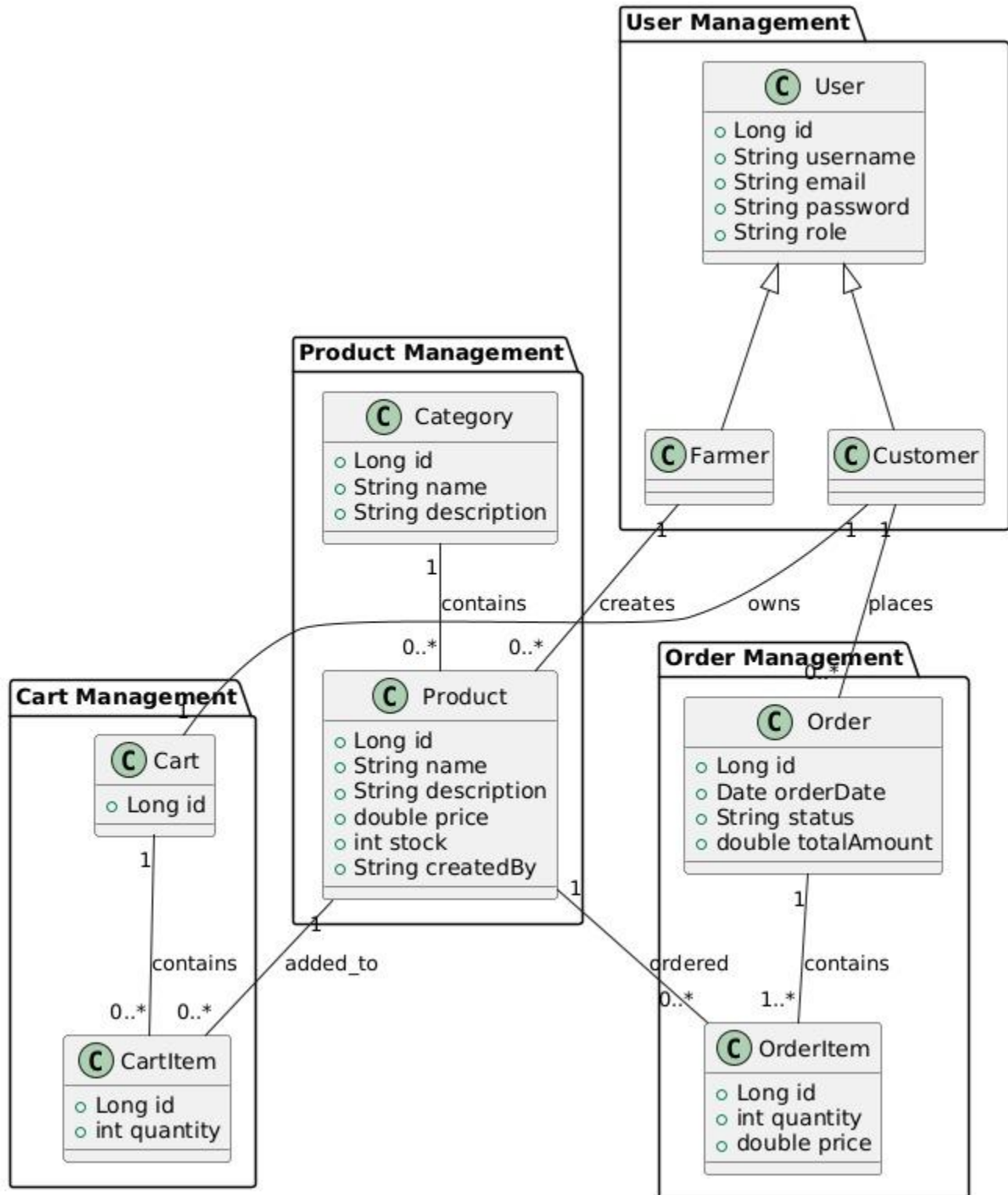
Level 2 :**OrganicBazar DFD - Level 2 (Order Management)**

4.4 Activity Diagram :

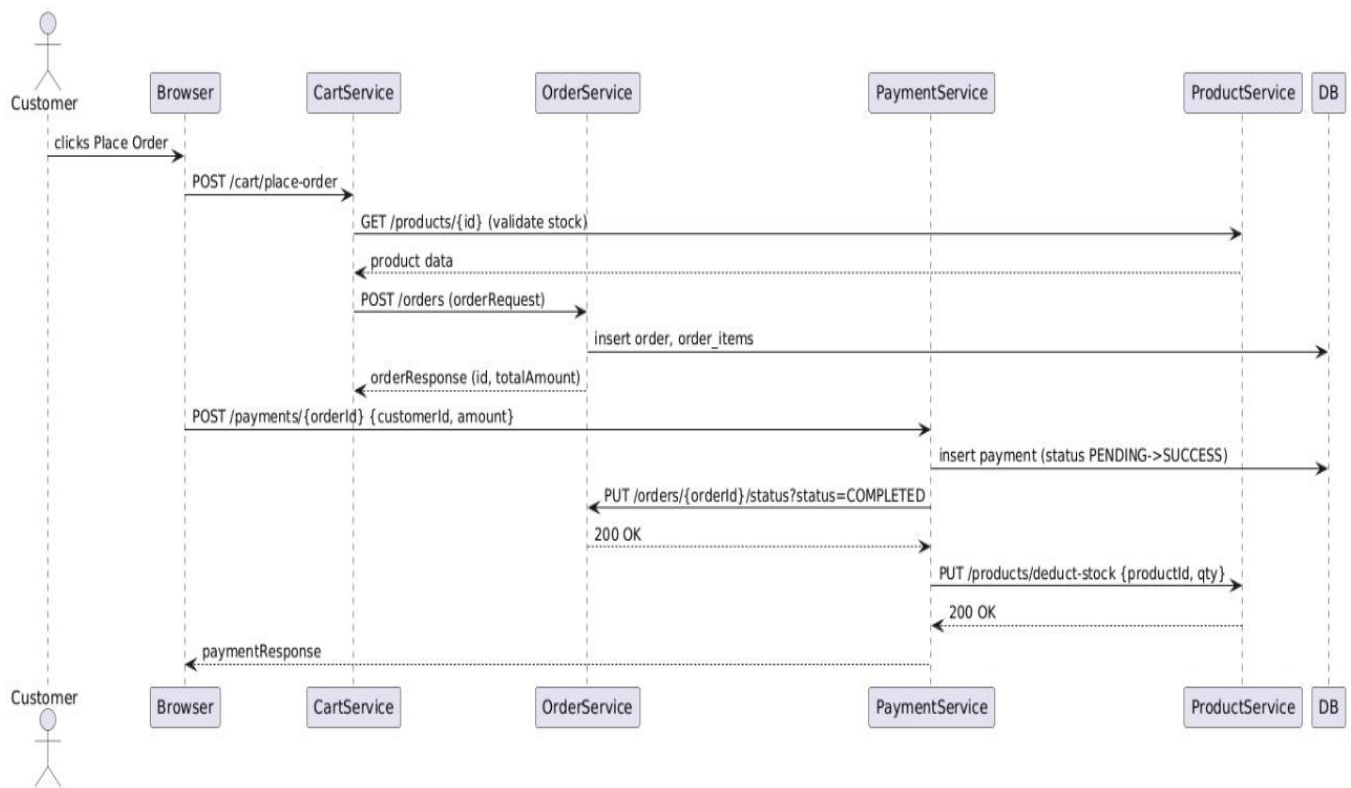


4.5 Class Diagram :

OrganicBazar Class Diagram

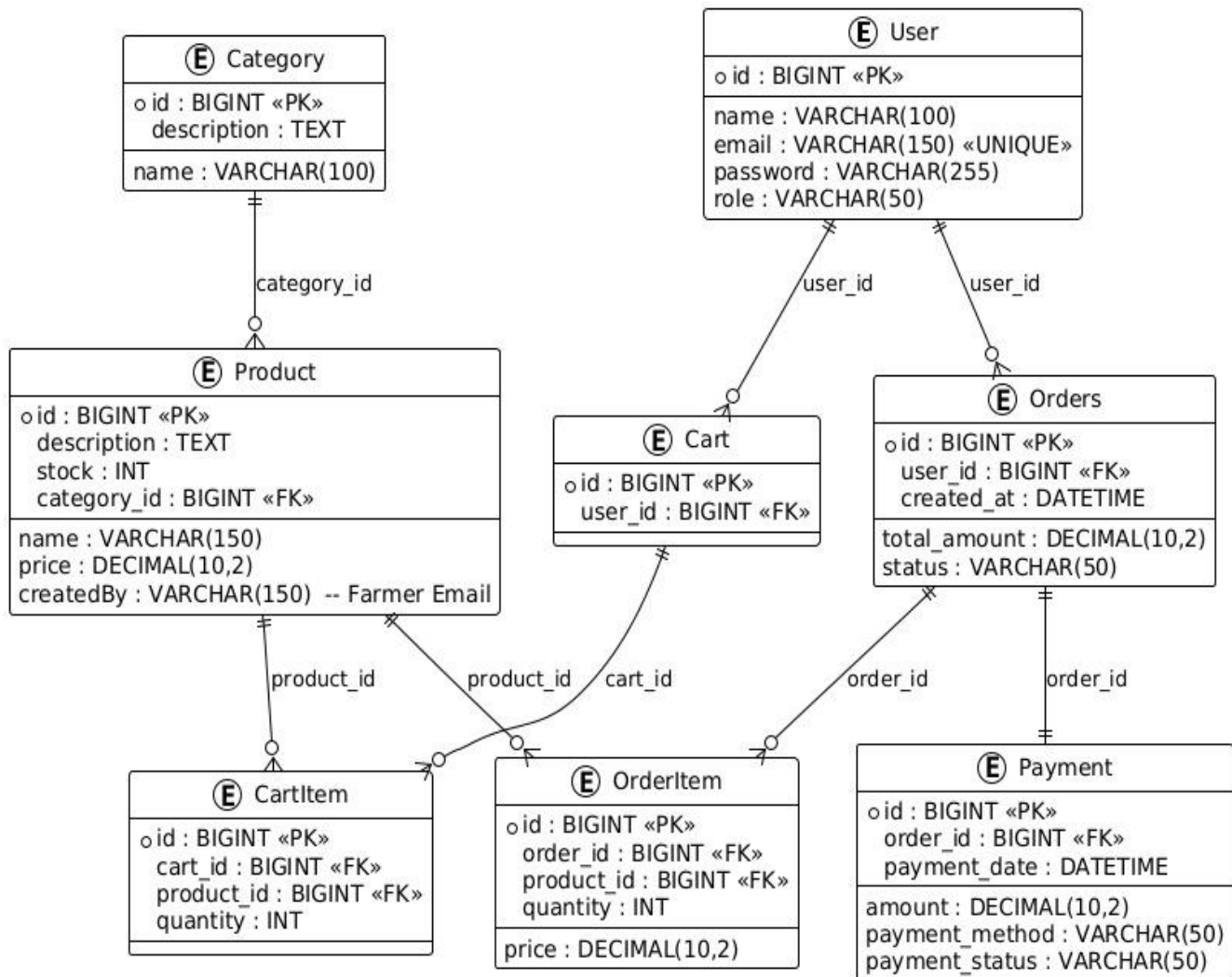


4.6 Sequence Diagram :



5. DATABASE DESIGN

5.1 Design :



5.2 Tables :

```
mysql> desc user;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
email	varchar(255)	YES		NULL	
password	varchar(255)	YES		NULL	
role	enum('ADMIN','FARMER','CUSTOMER')	YES		NULL	
username	varchar(255)	YES		NULL	

```
5 rows in set (0.01 sec)
```

```
mysql> desc product;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
category_id	bigint	YES		NULL	
category_name	varchar(255)	YES		NULL	
created_by	varchar(255)	YES		NULL	
description	varchar(255)	YES		NULL	
name	varchar(255)	YES		NULL	
price	double	NO		NULL	
stock	int	NO		NULL	

```
8 rows in set (0.00 sec)
```

```
mysql> desc category;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
description	varchar(255)	YES		NULL	
name	varchar(255)	YES		NULL	

```
3 rows in set (0.01 sec)
```

```
mysql> desc cart_item;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
customer_id	bigint	YES		NULL	
product_id	bigint	YES		NULL	
quantity	int	NO		NULL	

```
4 rows in set (0.00 sec)
```

```
mysql> desc order_items;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
product_id	bigint	YES		NULL	
quantity	int	YES		NULL	
order_id	bigint	YES	MUL	NULL	

4 rows in set (0.00 sec)

```
mysql> desc orders;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
customer_id	bigint	YES		NULL	
order_date	datetime(6)	YES		NULL	
status	varchar(255)	YES		NULL	
total_amount	double	YES		NULL	

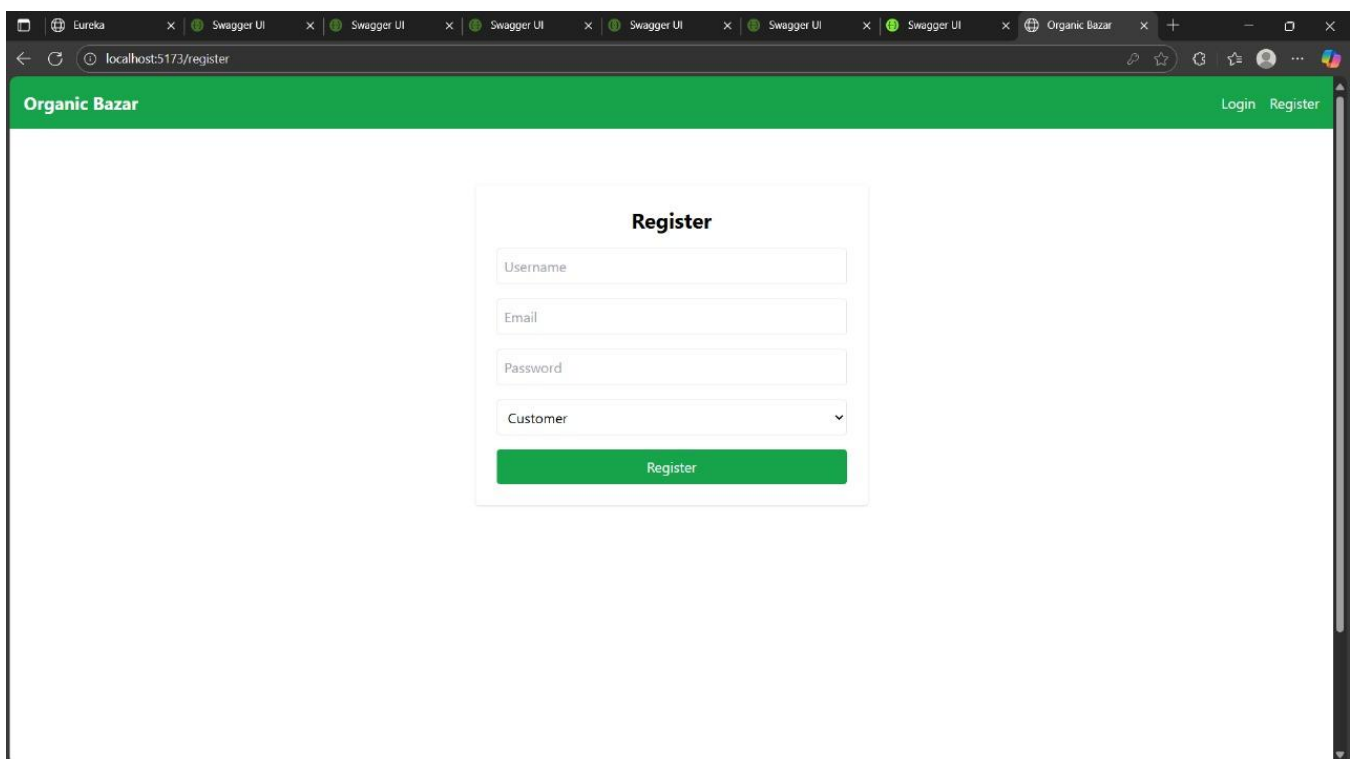
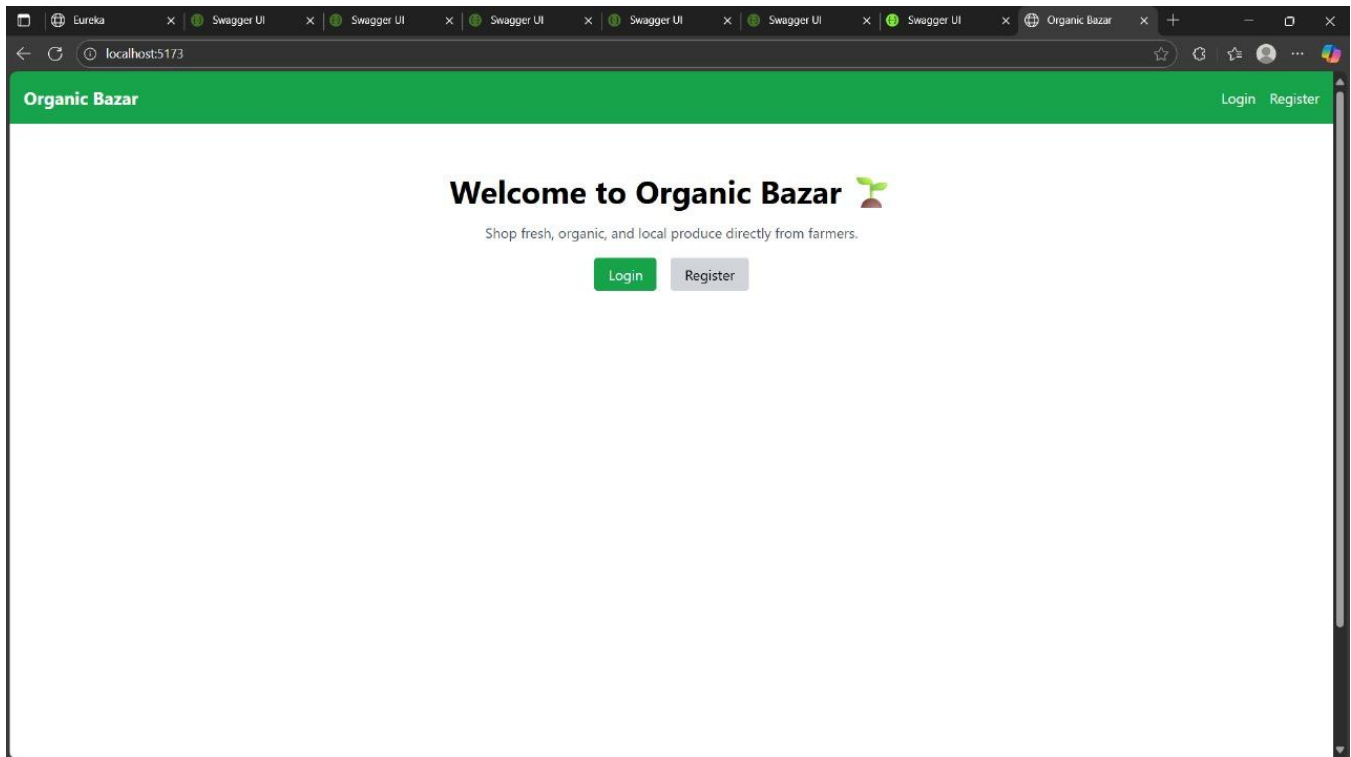
5 rows in set (0.00 sec)

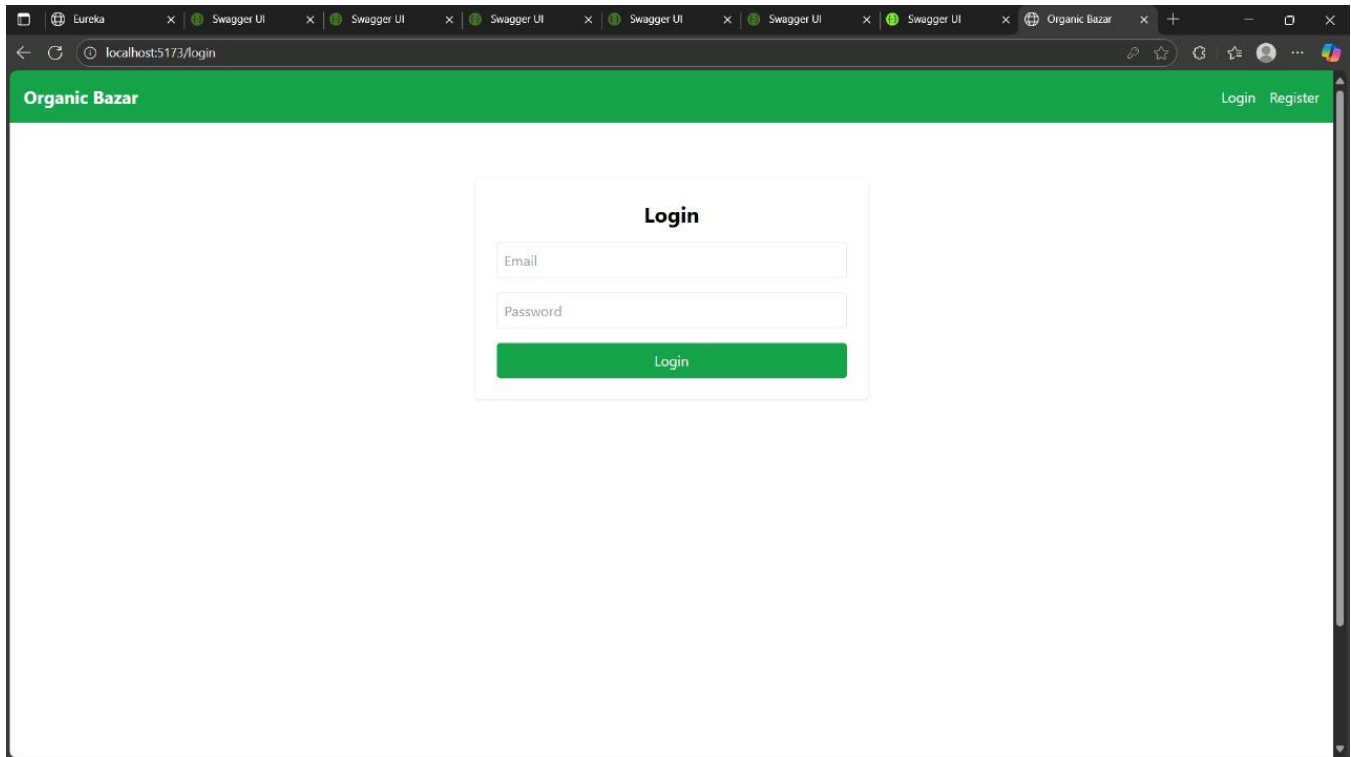
```
mysql> desc payment;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
amount	double	YES		NULL	
customer_id	bigint	YES		NULL	
method	varchar(255)	YES		NULL	
order_id	bigint	YES		NULL	
payment_time	datetime(6)	YES		NULL	
status	varchar(255)	YES		NULL	
payment_date	datetime(6)	YES		NULL	

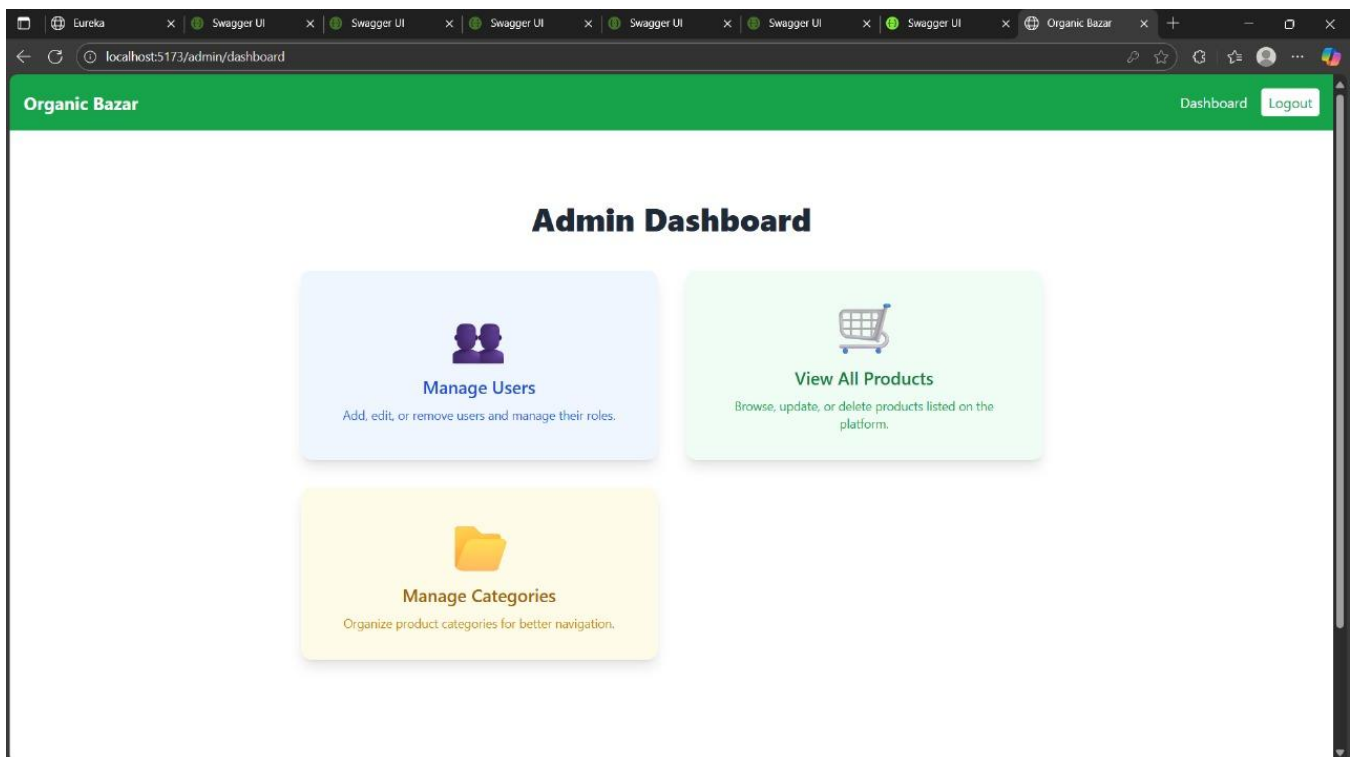
8 rows in set (0.00 sec)

6. SNAPSHOTS





The screenshot shows a web browser window with the URL `localhost:5173/login`. The page has a green header with the text "Organic Bazar" on the left and "Login Register" on the right. The main content area is white and contains a centered "Login" form. The form has a title "Login", an "Email" input field, a "Password" input field, and a green "Login" button.



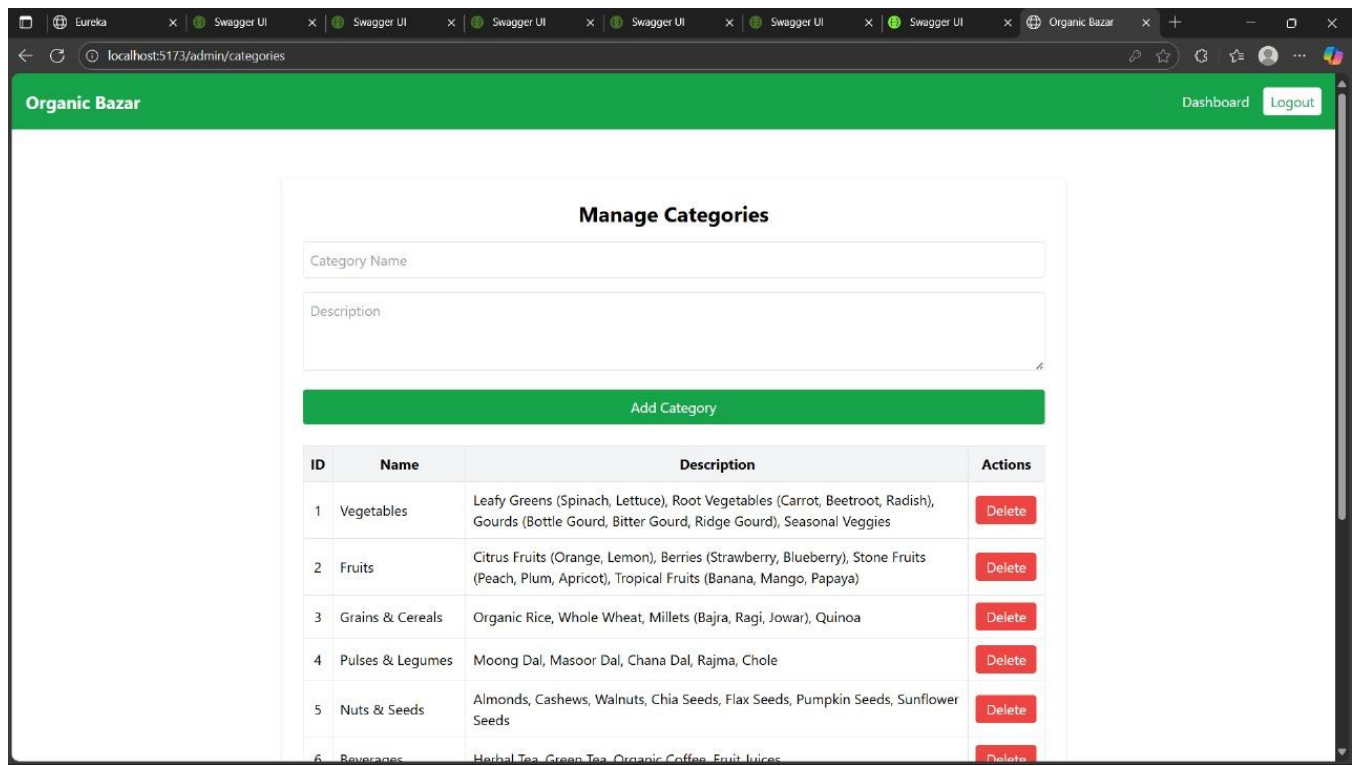
The screenshot shows a web browser window with the URL `localhost:5173/admin/dashboard`. The page has a green header with the text "Organic Bazar" on the left and "Dashboard Logout" on the right. The main content area is white and contains a centered "Admin Dashboard" section. This section has three cards: "Manage Users" (blue background, icon of two people, description: "Add, edit, or remove users and manage their roles."), "View All Products" (green background, icon of a shopping cart, description: "Browse, update, or delete products listed on the platform."), and "Manage Categories" (yellow background, icon of a folder, description: "Organize product categories for better navigation.").

The screenshot shows a web browser window with the URL `localhost:5173/admin/users`. The page has a green header bar with the text "Organic Bazar" on the left and "Dashboard" and "Logout" links on the right. Below the header, the page is titled "All Users". It contains a table with the following data:

ID	Username	Email	Role	Actions
1	admin	admin@gmail.com	ADMIN	Delete
2	farmer	farmer@gmail.com	FARMER	Delete
4	customer	customer@gmail.com	CUSTOMER	Delete

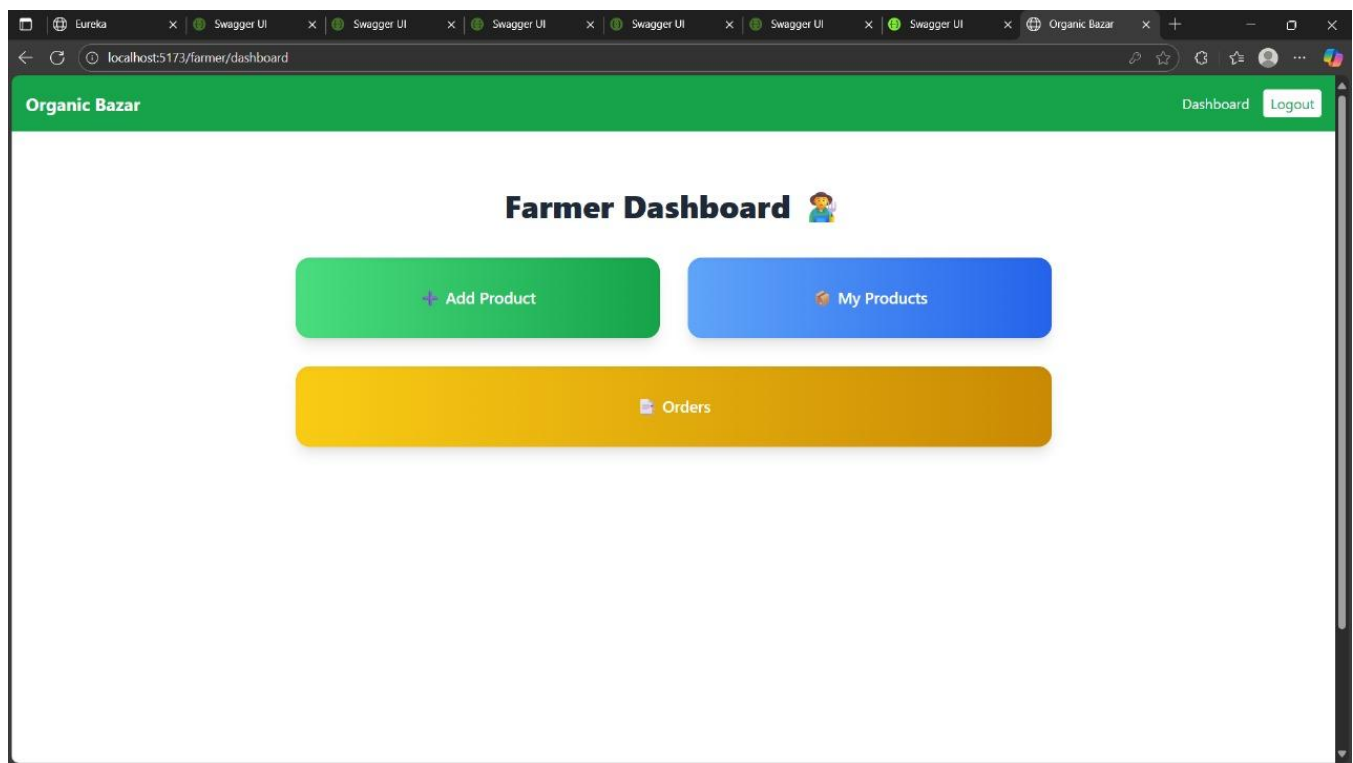
The screenshot shows a web browser window with the URL `localhost:5173/admin/products`. The page has a green header bar with the text "Organic Bazar" on the left and "Dashboard" and "Logout" links on the right. Below the header, the page is titled "All Products". It contains a table with the following data:

ID	Name	Price	Stock	Category
1	Apple	₹50	34	Fruits
2	Milk	₹50	29	Dairy & Alternatives
3	Tomato	₹20	22	Vegetables



The screenshot shows the 'Manage Categories' page of the Organic Bazar admin dashboard. The page has a green header with the 'Organic Bazar' logo and a 'Logout' button. The main content area contains a form for adding a new category with fields for 'Category Name' and 'Description', and a green 'Add Category' button. Below the form is a table listing existing categories with columns for ID, Name, Description, and Actions (Delete).

ID	Name	Description	Actions
1	Vegetables	Leafy Greens (Spinach, Lettuce), Root Vegetables (Carrot, Beetroot, Radish), Gourds (Bottle Gourd, Bitter Gourd, Ridge Gourd), Seasonal Veggies	Delete
2	Fruits	Citrus Fruits (Orange, Lemon), Berries (Strawberry, Blueberry), Stone Fruits (Peach, Plum, Apricot), Tropical Fruits (Banana, Mango, Papaya)	Delete
3	Grains & Cereals	Organic Rice, Whole Wheat, Millets (Bajra, Ragi, Jowar), Quinoa	Delete
4	Pulses & Legumes	Moong Dal, Masoor Dal, Chana Dal, Rajma, Chole	Delete
5	Nuts & Seeds	Almonds, Cashews, Walnuts, Chia Seeds, Flax Seeds, Pumpkin Seeds, Sunflower Seeds	Delete
6	Beverages	Herbal Tea, Green Tea, Organic Coffee, Fruit Juices	Delete



The screenshot shows the 'Farmer Dashboard' of the Organic Bazar. The page has a green header with the 'Organic Bazar' logo and a 'Logout' button. The main content area features the title 'Farmer Dashboard' with a farmer icon. Below the title are three large, rounded rectangular buttons: a green 'Add Product' button, a blue 'My Products' button, and a yellow 'Orders' button.

The screenshot shows a web browser window with the URL `localhost:5173/farmer/add-product`. The page has a green header with the text "Organic Bazar" and a "Logout" button. The main content area is titled "Add Product" and contains a form with the following fields: "Name", "Description", "Price", "Stock", and a dropdown menu labeled "-- Select Category --". A green "Submit" button is located below the form.

Organic Bazar [Dashboard](#) [Logout](#)

Add Product

Name

Description

Price

Stock

-- Select Category --

[Submit](#)

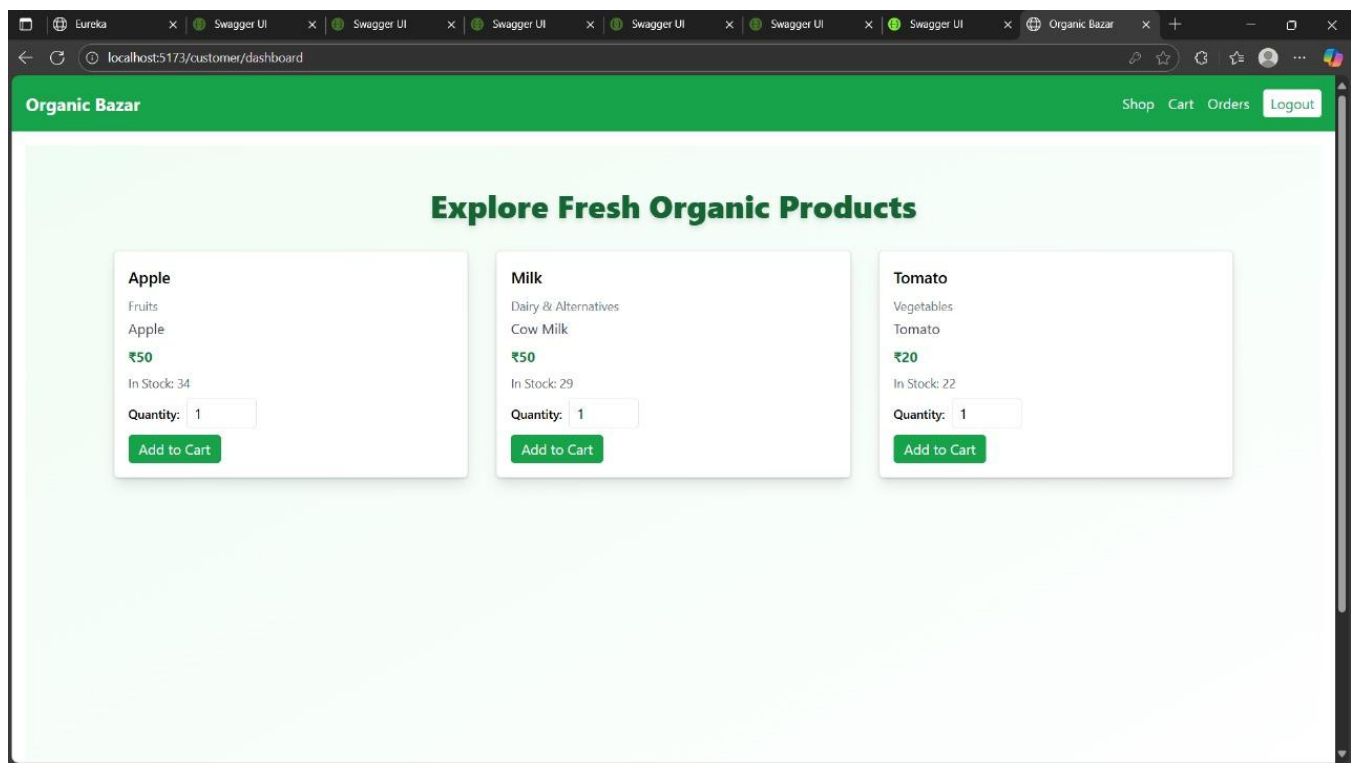
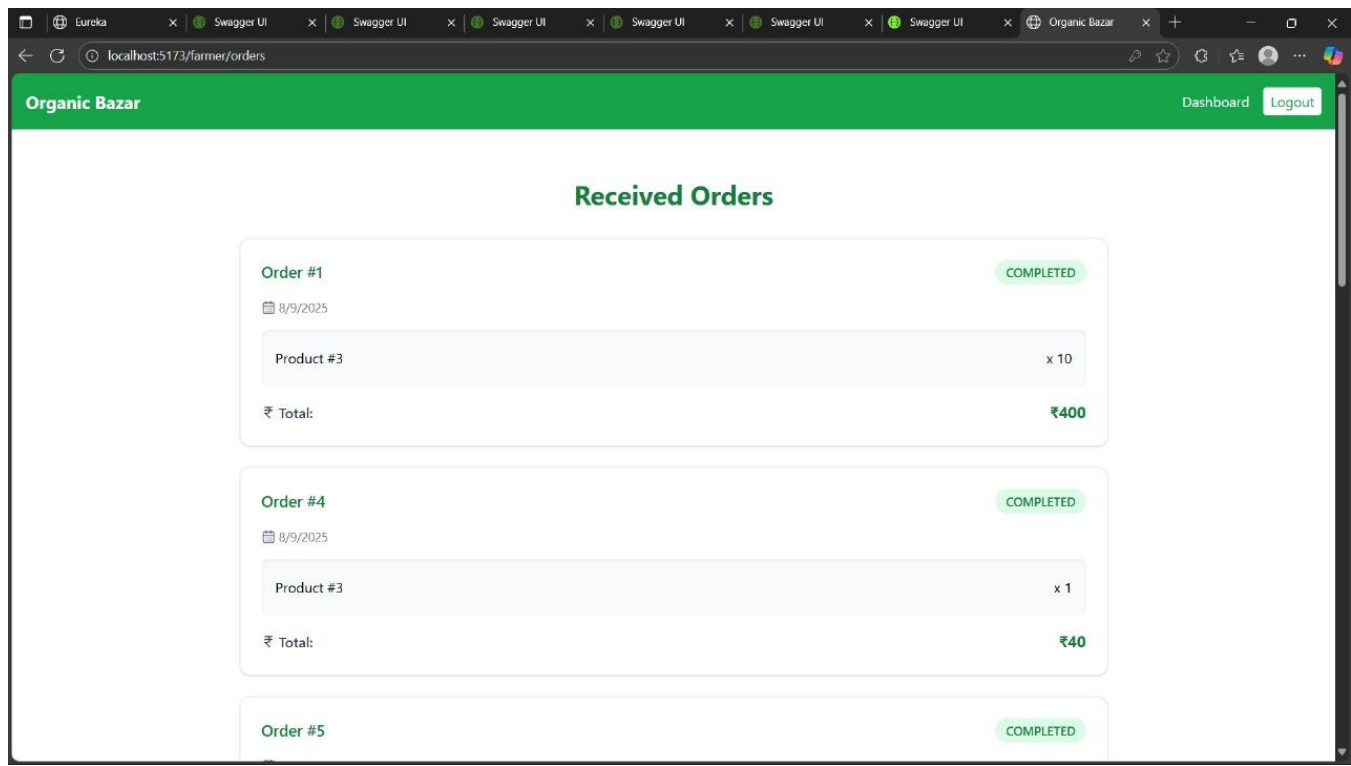
The screenshot shows a web browser window with the URL `localhost:5173/farmer/my-products`. The page has a green header with the text "Organic Bazar" and a "Logout" button. The main content area is titled "My Products" and contains a table with the following data:

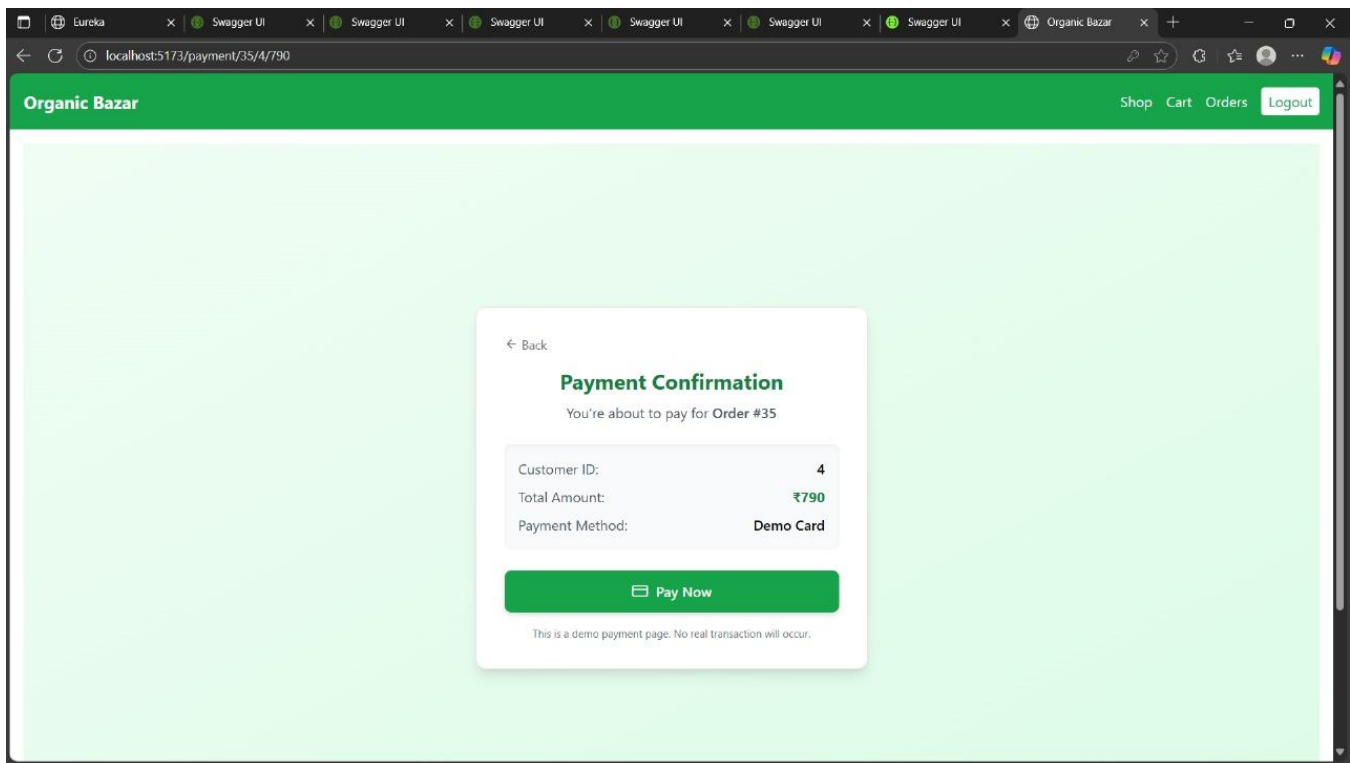
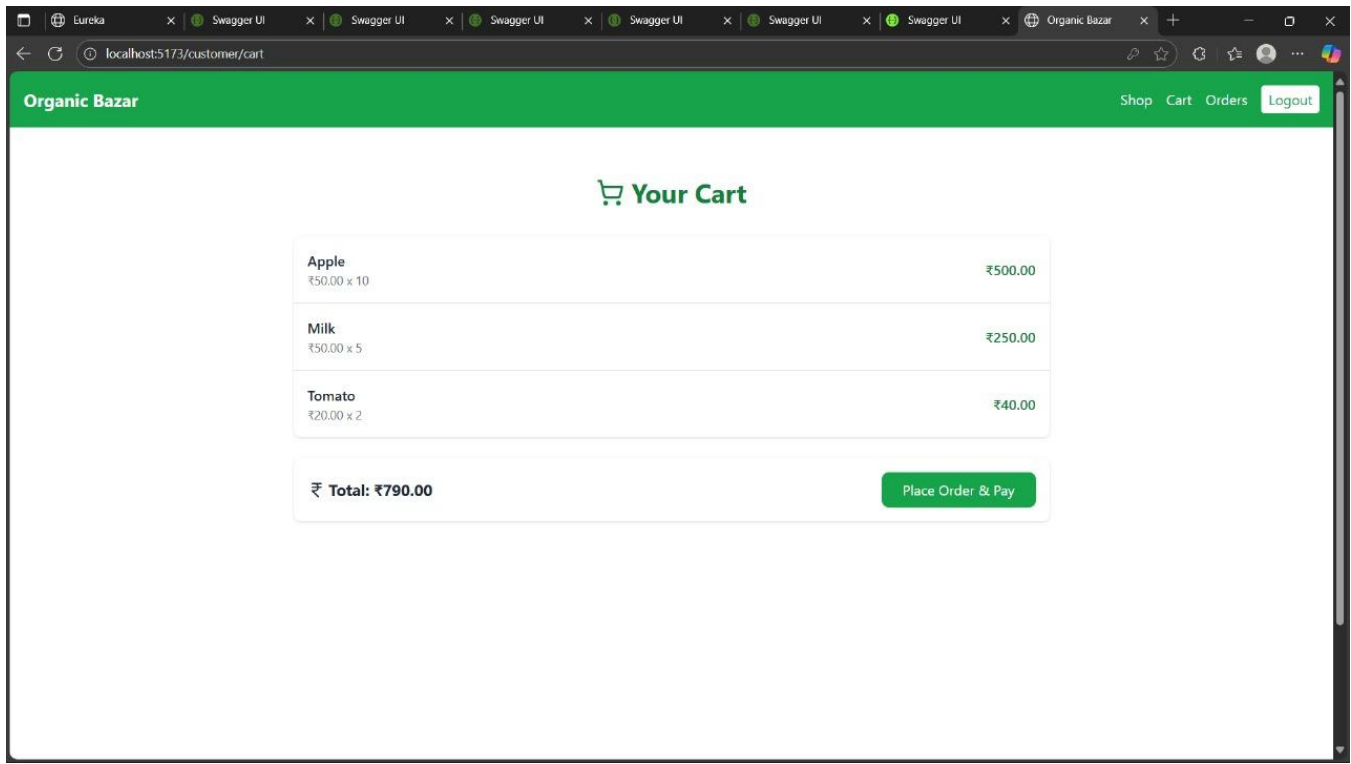
NAME	PRICE	STOCK	CATEGORY	ACTIONS
Apple	₹50	34	Fruits	Update Delete
Milk	₹50	29	Dairy & Alternatives	Update Delete
Tomato	₹20	22	Vegetables	Update Delete

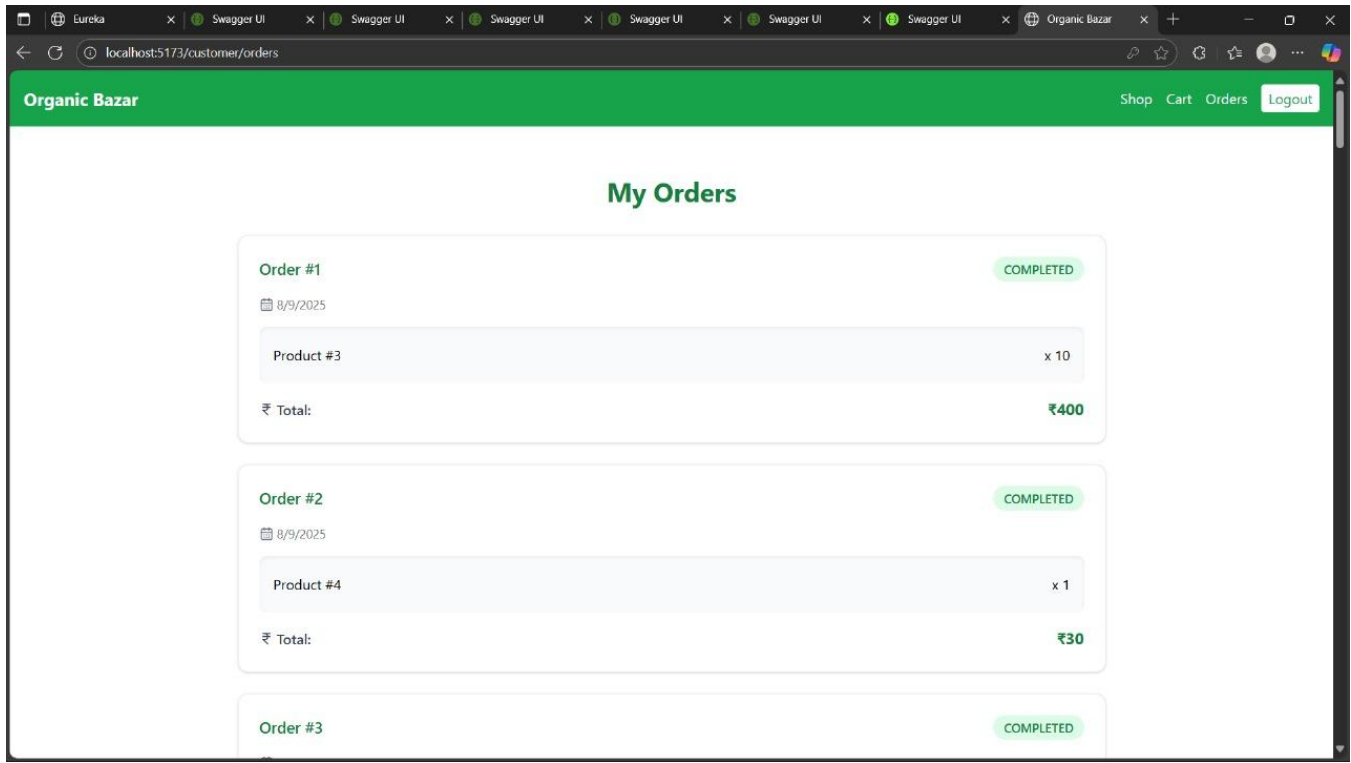
Organic Bazar [Dashboard](#) [Logout](#)

My Products

NAME	PRICE	STOCK	CATEGORY	ACTIONS
Apple	₹50	34	Fruits	Update Delete
Milk	₹50	29	Dairy & Alternatives	Update Delete
Tomato	₹20	22	Vegetables	Update Delete







7. CONCLUSION

The **Organic Bazar** project successfully delivers a robust and user-friendly platform that bridges the gap between organic product suppliers and consumers through an efficient online marketplace. By integrating secure authentication, intuitive navigation, advanced product filtering, cart management, and streamlined order processing, the system ensures a seamless experience for all stakeholders, including customers, farmers, and administrators.

The platform not only enhances the accessibility of organic products but also promotes sustainable and healthy living by connecting consumers directly with trusted suppliers. Its modular architecture, built using **Spring Boot** for the backend and **React JS** for the frontend, ensures scalability, maintainability, and adaptability to future enhancements.

Through its comprehensive functionality and adherence to best development practices, **Organic Bazar** stands as a reliable, transparent, and impactful solution in the growing domain of organic e-commerce, paving the way for future expansion and feature integration.

8. FUTURE SCOPE

The Organic Bazar application has significant potential for future enhancements to meet evolving market needs and technological advancements. In the future, the platform can be integrated with AI-powered product recommendations and personalized offers based on user preferences and buying history. Blockchain technology can be introduced to ensure transparency and traceability of organic products, building customer trust. The system can also incorporate advanced analytics for farmers and sellers to optimize inventory, pricing, and sales strategies. Additionally, expanding the application into a multilingual platform with support for multiple currencies will make it accessible to a wider audience. Features such as subscription-based delivery services, integration with IoT devices for real-time farm updates, and partnerships with logistics providers for faster and eco-friendly delivery can further enhance the user experience and market reach.

9. REFERENCES

1. <https://docs.spring.io/spring-security/reference/>
2. <https://docs.spring.io/spring-boot/index.html/>
3. Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). *Database Systems: The Complete Book*. Prentice Hall.
4. Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377-387.
5. Fowler, M. (2003). *Patterns of Enterprise Application Architecture*. Addison- Wesley.
6. Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
7. Soni, D., Nord, R. L., & Hofmeister, C. (1995). *Software Architecture in Industrial Applications*. Proceedings of the 17th International Conference on Software Engineering, 196-207.